

Project Documentation: Twitter Sentiment Analysis System

System Version: 1.0.0 (Production Stable)

1. System Architecture Overview

This project implements a complete, interactive, machine learning-driven text classification framework designed to analyze public sentiment from short textual elements (tweets). The system consists of two explicit subsystems:

- 1. The Model Training Pipeline (Jupyter Notebook):** Handles deterministic ingestion of data, structural text preprocessing (regular expressions, tokenization, stopwords elimination, and Porter stemming), high-dimensional numerical feature extraction using Term Frequency-Inverse Document Frequency (TF-IDF) vectorization, and model minimization via Logistic Regression.
- 2. The Web User Interface (Streamlit Application):** Provides a reactive production environment loading saved pipeline components dynamically to classify arbitrary live strings or aggregate individual users' past entries directly from the dataset records.

```
[Raw Data: tweeter-dataset.zip] → [Regex & NLTK Stemming] → [TF-IDF Vectorizer Fit]
    → [Logistic Regression Model] → [Export: model.pkl & vectorizer.pkl]
    → [Streamlit Web UI (app.py Inference Engine)]
```

2. Model Training Pipeline Execution (sentimental-analysis.ipynb)

The code blocks within the core notebook manage sequential feature modeling, formatting historical input records into standardized structures acceptable for production. Below is a detailed section-by-section mathematical and logic layout explanation.

2.1 Dependency Isolation & Core Initializations

```
import numpy as np
import pandas as pd
import re
import nltk
import pickle
from nltk.corpus import stopwords
from nltk.stem.porter import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

nltk.download('stopwords')
stemmer = PorterStemmer()
stop_words = set(stopwords.words('english'))
```

Technical Analysis: This initialization step imports foundational requirements for mathematical array handling, dataframe modifications, and serializing operations. It loads standard text optimization dependencies from NLTK, downloading and compiling the localized stopwords list. This drops highly recurrent non-emotional words (e.g., "the", "is", "an") to save computational overhead during matrix transformations.

2.2 Class-Balanced Dataset Loading

```
dataset_path = r"G:\Jupyter-NoteBook\sentimental-analysis\weeter-dataset.zip"
print("⌚ Loading dataset...")
raw_data = pd.read_csv(dataset_path, compression='zip', encoding='latin-1', header=None)

col_names = ['target', 'ids', 'date', 'flag', 'user', 'text']
raw_data.columns = col_names
raw_data['target'] = raw_data['target'].map({4: 1, 0: 0})

negative_tweets = raw_data[raw_data['target'] == 0]
positive_tweets = raw_data[raw_data['target'] == 1]

data_negative_segment = negative_tweets.iloc[:2500]
data_positive_segment = positive_tweets.iloc[:2500]

data = pd.concat([data_negative_segment, data_positive_segment]).copy()
```

Technical Analysis: Streams data rows from a zipped location using `latin-1` character decoding to gracefully capture complex social media strings without crashing. The raw 0-to-4 metrics are standard binary-mapped to `[0, 1]` for target alignment. To provide a predictable, unskewed baseline for optimization testing, the logic sequentially grabs exactly 2,500 negative rows and 2,500 positive rows, forming a clean 5,000-row dataframe slice.

2.3 Text Cleansing & Algorithmic Stemming Function

```
def stemming(content):
    stemmed_content = re.sub('[^a-zA-Z]', ' ', content)
    stemmed_content = stemmed_content.lower()
    stemmed_content = stemmed_content.split()
    stemmed_content = [stremmer.stem(word) for word in stemmed_content if word not in
stop_words]
    return ' '.join(stemmed_content)

data['text'] = data['text'].apply(stemming)
```

Technical Analysis: Standardizes input data features. The regex pattern `[^a-zA-Z]` cleanses text strings by swapping out non-alphabetic elements like punctuation and numeric values for clean blank spaces. Content strings are case-normalized to lower-case blocks, split into standalone tokens, and passed through the `PorterStemmer` framework after filtering out uninformative stopwords. This normalizes related linguistic inflections (e.g., "studying", "studies", and "studied" are all stemmed to `"studi"`).

2.4 Dataset Partitioning & Stratified Validation

```
x = data['text']
y = data['target']

x_train_raw, x_test_raw, y_train, y_test = train_test_split(
    x, y,
    test_size=0.2,
    shuffle=True,
    stratify=y,
    random_state=42
)
```

Technical Analysis: Allocates 20% of the active dataset (1,000 observations) strictly for model validation testing, reserving 4,000 elements for training. The configuration `stratify=y` guarantees that both training and testing subsets inherit identical target class splits, preventing selection bias. The `random_state=42` seed locks structural generation boundaries for clean code reproducibility across execution runs.

2.5 TF-IDF Feature Extraction & Numerical Vectorization

```
vectorizer = TfidfVectorizer()
x_train = vectorizer.fit_transform(x_train_raw)
x_test = vectorizer.transform(x_test_raw)
```

Technical Analysis: Machine learning algorithms require consistent numerical arrays rather than raw text blocks. This script configures a `TfidfVectorizer` matrix to convert text tokens into statistical weight mappings.

****Crucially****, it runs `fit_transform()` solely across the training subset to build the system dictionary. The testing partitions are structural-mapped via `.transform()` to completely prevent test-data leakage.

2.6 Logistic Regression Optimization & Production Serialization

```
model = LogisticRegression(max_iter=1000)
model.fit(x_train, y_train)

with open('model.pkl', 'wb') as model_file:
    pickle.dump(model, model_file)
with open('vectorizer.pkl', 'wb') as vec_file:
    pickle.dump(vectorizer, vec_file)
```

Technical Analysis: Solves classification targets using a binary `LogisticRegression` model block. Setting `max_iter=1000` gives the numerical optimizer enough iterations to reach proper convergence without throwing termination warnings. The model achieves an internal testing accuracy score of ****70.40%****. Once training completes, `pickle` serializes the active model and feature structures into portable files (`model.pkl` and `vectorizer.pkl`) for web layer integration.

Figure 1: Jupyter Notebook Execution Console Logs

Visualizes sequential dataset preprocessing metrics, the verified model performance of 70.40%, and successful production artifact exports.

3. Production Application Layer Deployment (app.py)

The web layer imports the serialized analytical artifacts into an interactive framework. The script couples Streamlit's layout engine with custom HTML styling rules to execute fast manual testing and historical user queries.

3.1 Streamlit Performance Optimization Mechanics

- **Memory Performance via `@st.cache_resource`** : Heavy analytical dependency initializations (like loading saved models and the NLTK data array) run exactly once. The framework caches these resources in persistent memory, providing instant sub-millisecond retrieval on application re-renders instead of triggering slow, repetitive file reads.
- **Data Profiling via `@st.cache_data`** : Streams the main dataset archive into local RAM. This structure lets the web system process lookup matching queries instantly without experiencing file parsing bottlenecks during interactive lookups.
- **Custom Dynamic Interface Card Components:** Replaces standard plaintext output messages with clean, color-coded HTML status boxes. This interface injects conditional styling configurations directly into the rendering tree based on the classification results.

Figure 2: Streamlit Application Script Core Environment Map

Illustrates asset streaming pathways, global caching tiers, and interactive user lookup logic routes.

4. Interface Outputs & Classification Mockups

This section showcases the functional application screens generated by the web interface. The portal renders responsive, targeted layouts based on the chosen mode selection.

4.1 Mode A: Real-Time Custom Text Ingestion

When users enter natural language strings into the input zone, the application processes the text through the cleaning pipeline and displays a color-coded sentiment breakdown.

Streamlit Web Portal — Free-form Classification Mode

Choose an option

Input text

Enter text to analyze sentiment

I am absolutely thrilled with this new development stack, it runs incredibly fast!

Analyze

POSITIVE SENTIMENT

I am absolutely thrilled with this new development stack, it runs incredibly fast!

Streamlit Web Portal — Free-form Classification Mode (Negative Class Response)

Enter text to analyze sentiment

This product is completely broken. It crashed during initialization and wasted my afternoon.

Analyze

NEGATIVE SENTIMENT

This product is completely broken. It crashed during initialization and wasted my afternoon.

4.2 Mode B: Automated Database User Profile Queries

When selecting historical lookups, the interface queries the indexed data rows for username matches, isolates up to five recent entries, and updates the application window with a structured text overview dashboard.

Streamlit Web Portal — Historical Database User Search Profile

Choose an option

Get tweets from user

Enter Twitter username

scotthamilton

Fetch Tweets

✓ Found 5 records for user: @scotthamilton

NEGATIVE SENTIMENT
upset updat facebook text might cri result school today...

NEGATIVE SENTIMENT
my head hurts so bad from studying all night long without breaks.

5. Engineering Verification & Troubleshooting Registry

Observed Error	Root Structural Cause	Remediation Steps
<code>FileNotFoundError:</code> <code>tweeter-dataset.zip</code>	The relative folder path tracking link is broken or file records were extracted incorrectly.	Ensure the target dataset file sits in the same active directory folder level as your running script thread.
<code>AttributeError:</code> <code>Vectorizer lacks</code> <code>vocabulary_</code>	The vectorizer object was exported to disk before completing vocabulary initialization steps.	Re-run your pipeline notebook completely. Verify that the <code>fit_transform()</code> phase finishes before exporting objects.
Slow execution on entry lookups	The NLTK stopword dataset is reloading and checking disk records during every internal prediction cycle loop step.	Wrap your asset-loading operations inside Streamlit's <code>@st.cache_resource</code> decorator block to keep dependencies in active RAM.